Documentação Técnica: Pipeline de Dados End-to-End

1. Visão Geral e Contexto de Negócio

Este projeto implementa uma arquitetura de dados escalável para suportar a tomada de decisão em tempo real. O pipeline extrai dados de sistemas dispersos, centraliza-os numa base operacional (Neon PostgreSQL) e transforma-os através de processamento distribuído (Databricks/PySpark), culminando numa semântica de dados otimizada (Delta Lake) para consumo via Power BI e Databricks Dashboards.

O foco principal desta arquitetura é garantir **resiliência na ingestão**, **qualidade de dados na transformação** e **alta performance no consumo** por parte da equipa de negócio.

2. Arquitetura de Dados e Tech Stack

O pipeline foi construído utilizando as melhores práticas de Engenharia de Dados, separando a camada operacional da camada analítica:

- **Fontes de Origem:** APIs REST, Azure (Blob Storage/SQL) e transações originadas no próprio Neon PostgreSQL.
- **Ingestão de Dados:** Python e Pandas (Script de ETL).
- **Armazenamento Operacional (Stage/OLTP):** Neon PostgreSQL.
- **Processamento Distribuído:** Databricks (PySpark).
- **Data Warehouse / Lakehouse:** Delta Lake (Camada Gold).
- **Orquestração e Automação:** GitHub Actions (CI/CD) e Databricks Jobs.
- **Visualização e BI:** Power BI (via DirectQuery) e Databricks Lakeview.

3. Fluxo de Dados: Camada a Camada

3.1. Fase de Ingestão (Extract & Load)

O módulo de ingestão, desenvolvido em **Python** (`etl_pipeline.py`), é responsável por recolher os dados brutos de três fontes distintas:

1. **APIs e Web Services:** Extração de dados externos utilizando a biblioteca `requests`, com implementação de paginação e tratamento de erros de conexão (Timeouts e HTTP 429).

2. **Azure Blob Storage / SQL:** Recolha de ficheiros históricos e logs estruturados em nuvem.

3. **Neon PostgreSQL (Fonte Transacional):** Captura das transações diárias operacionais.

Destino: Todos os dados extraídos são consolidados num repositório centralizado de *stage* no **Neon PostgreSQL**. Foi utilizada a biblioteca `SQLAlchemy` / `psycopg2` com a abordagem de *Upsert* (Update/Insert) para evitar dados duplicados nas cargas incrementais diárias.

3.2. Fase de Processamento e Transformação (Databricks PySpark)

A carga pesada ocorre no ambiente Databricks, que se conecta à base Neon via conexão **JDBC**. Foram aplicadas as seguintes regras de engenharia no PySpark para garantir a robustez da Camada Gold:

- **Limpeza e Normalização:**
-
    - **Tratamento de Nulos:** Remoção de registos sem chaves primárias e preenchimento de dimensões vazias com valores padrão ("N/A" ou "Desconhecido").
    -
    - **Conversão de Tipos (Casting):** Padronização rigorosa de formatos de data (`TimestampType`) e valores monetários (`DecimalType`) para otimização do motor SQL.
    -
- **Lógica de Negócio e Consolidação:**
    - Cruzamento (Join) dos dados comportamentais vindos da API com os dados transacionais do Neon e Azure.
    - Uso de *Window Functions* no PySpark para deduplicação, mantendo apenas a interação mais recente de cada entidade.

3.3. Fase Analítica e Consumo (Delta Lake & BI)

- **Delta Lake (Gold Tier):** Os dados limpos e transformados são guardados no formato `.delta`. Isto garante transações ACID, proteção contra corrupção de dados e compressão otimizada em Parquet.

- **Power BI:** O consumo é feito através do motor Databricks SQL (Serverless) via **DirectQuery**, garantindo que os relatórios refletem o estado mais recente dos dados sem necessidade de duplicação ou importações pesadas no ficheiro `.pbix`.

4. Orquestração e Automação (CI/CD)

A arquitetura opera num modelo *Zero-Touch* (sem intervenção manual), orquestrada em dois níveis:

1. **GitHub Actions:**

    - Gere a implementação do código e executa o script de ingestão Python.
    - Agendamento via *Cron Expression* para rodar de madrugada (ex: 02:00 AM UTC).
    - *Segurança:* Nenhuma credencial está no código. As chaves da API, do Azure e do Neon são injetadas no ambiente de execução de forma segura via **GitHub Secrets**.

2. **Databricks Jobs:**

    - Agendado para executar o *Notebook* PySpark logo após a conclusão do fluxo no GitHub Actions. Provisiona recursos computacionais de forma efémera e desliga-se ao terminar, otimizando os custos da cloud.
    -

5. Troubleshooting Operacional (Runbook)

Caso ocorra alguma falha na atualização diária, o fluxo de resolução deve ser:

1. **GitHub Actions (Falha na Ingestão):**

    - Aceder ao separador *Actions* no repositório. O erro mais comum é limite de *requests* na API de origem ou alteração de *passwords*.

2. **Databricks (Falha na Transformação):**

    - Aceder a *Workflows -> Job Runs*. Verificar se houve quebra no esquema de origem (*Schema Drift*) da base de dados Neon que impeça a leitura via JDBC.

3. **Power BI (Dashboard Desatualizado):**

    - Confirmar no Power BI Service se a credencial *Personal Access Token (PAT)* do Databricks continua válida e se o *Databricks SQL Warehouse* está ativo.

6. Como Executar o Projeto Localmente

Para clonar e testar a ingestão no teu ambiente de desenvolvimento (VS Code):

Bash

None

```
# 1. Clonar o repositório
git clone https://github.com/teu-usuario/nome-do-repositorio.git
cd nome-do-repositorio

# 2. Criar ambiente virtual e instalar dependências
python -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\activate
pip install -r requirements.txt

# 3. Configurar Variáveis de Ambiente
# Cria um ficheiro .env na raiz do projeto com as credenciais (não enviar para o Git)
API_KEY=***
AZURE_CONN_STRING=***
NEON_DB_URL=***

# 4. Executar o Pipeline de Ingestão
python etl_pipeline.py
```